FLAT PYQ Solutions

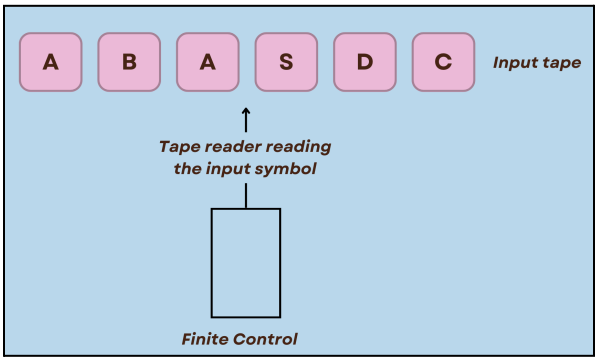
Section - A

Q1. Define the string. *(2 Marks)*

A string is a finite sequence of symbols selected from a given alphabet set. It represents a way to arrange symbols linearly. Strings can be empty (denoted by ε) or have multiple characters. For example, over Σ = {0,1}, "0101" is a string.

Q2. Draw a finite automata model. *(2 Marks)*

A finite automaton is a mathematical model used to recognize patterns within input data. It consists of a finite set of states, transitions between states based on input symbols, an initial state, and one or more accepting states.



Q3. Rewrite the meaning of epsilon transition. *(2 Marks)*

An epsilon (ε) transition in automata theory allows a machine to move from one state to another **without consuming any input symbol**. It enables flexibility by allowing free moves between states. Epsilon transitions are important in the conversion between NFA and DFA.

Q4. Restate the role of Read/Write head in the Turing machine. *(2 Marks)*

The Read/Write head in a Turing Machine moves over an infinite tape, reading the symbol under it, writing a new symbol (if needed), and moving left or right based on the machine's transition function. It controls the flow of computation by modifying the tape contents and shifting position during each transition step.

Q5. State the statement of Arden's theorem. *(2 Marks)*
Arden's Theorem provides a method to solve equations involving regular expressions. It states that if an equation is of the form P = Q + PR, then the solution is P = QR*. This theorem is fundamental in finding regular expressions from finite automata automatically and is widely used in formal language theory.

Q6. Enlist the applications of Context-Free Languages. *(2 Marks)*
Applications of Context-Free Languages include:

- Programming languages syntax parsing
- Compilers and interpreters
- XML document validation
- Arithmetic expression evaluation

Q7. Discuss derivation and derivation/parse tree in short. (2 Marks)
Derivation is the sequence of applying production rules to generate strings from a grammar's start symbol. A parse tree visually represents this derivation, showing how the start symbol expands into a string. It illustrates the hierarchical structure of a sentence or expression according to grammar rules.

Q8. Define Ambiguous grammar. (2 Marks)
A grammar is said to be ambiguous if there exists at least one string that can have **more than one leftmost or rightmost derivation** or **more than one parse tree**. Ambiguity creates confusion in interpretation, making it difficult for compilers and parsers to process strings uniquely.

Q9. Write short note on Church's Hypothesis. (2 Marks)
Church's Hypothesis, also known as the Church-Turing thesis, states that any function that can be computed algorithmically can also be computed by a Turing machine. It establishes the foundation for the theory of computation, connecting algorithms, effective procedures, and Turing machines under one concept.

Q10. State the meaning of Unit productions. (2 Marks)

A unit production in a grammar is a production rule where a single non-terminal produces another single non-terminal, like $A \rightarrow B$. These productions do not directly generate terminals and are often removed during grammar simplification to make parsing more efficient.

Q11. Identify the tuple formation of the Turing Machine. (2 Marks)

A Turing Machine is formally represented as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

where Q = set of states, Σ = input alphabet, Γ = tape alphabet, δ = transition function, q_0 = initial state, B = initial tape symbol, and F = set of final states.

Q12. Define P and NP problem. (2 Marks)

P is the class of decision problems that can be solved in polynomial time by a deterministic Turing machine. NP is the class of decision problems whose solutions can be verified in polynomial time by a deterministic Turing machine. The famous P vs NP question asks whether every problem whose solution can be verified quickly can also be solved quickly.

Section - B

Q13. Differentiate between Moore and Mealy machines. (4 Marks)

Aspect	Moore Machine	Mealy Machine
Output dependency	Output depends only on the **current state**.	Output depends on the **current state and input**.
Output timing	Output is associated with states.	Output is associated with transitions.
State diagram complexity	Generally simpler, as outputs are fixed for states.	May have a more complex diagram due to transition outputs.

Speed	Slower to respond because output changes only after state changes.	Faster to respond to inputs because output changes immediately with input.
Representation	Each state is labeled with an output.	Each transition is labeled with an output.

Example:

- **Moore Machine:**
  Transition: (q0, 0) → q1; Output: '1' at q1

- **Mealy Machine:**
  Transition: (q0, 0/1) → q1 (input 0, output 1)

Thus, Moore machines are simpler but slower, while Mealy machines are more responsive.

Q16. Analyse the two normal forms of Context-Free Grammar. (4 Marks)

The two important normal forms of Context-Free Grammar (CFG) are:

1. **Chomsky Normal Form (CNF):**
   - Every production is of the form:
     A → BC or A → a (where A, B, C are non-terminals and 'a' is a terminal).
   - No ε-productions (except possibly for start symbol).
   - CNF simplifies parsing algorithms like CYK.
2. **Greibach Normal Form (GNF):**
   - Every production is of the form:
     A → aα (where 'a' is a terminal and α is a (possibly empty) string of non-terminals).
   - GNF is helpful in designing efficient parsers, especially for top-down parsing without backtracking.

Comparison:

- CNF helps in bottom-up parsing and proofs like pumping lemma for CFLs.
- GNF is used in top-down parsing where each step immediately consumes one input symbol.

Both forms reduce ambiguity and are essential for theoretical analysis and parser construction.

Q17. Illustrate Post Correspondence Problem with example. (4 Marks)

Post Correspondence Problem (PCP):
Given two lists of strings A = (a_1, a_2, ..., a_n) and B = (b_1, b_2, ..., b_n) over some alphabet, the problem is to find a sequence of indices i_1, i_2, ..., i_k such that:
$\mathbf{a_{i_1} a_{i_2} \ldots a_{i_k} = b_{i_1} b_{i_2} \ldots b_{i_k}}$

Example:
A = (1, 10, 11)
B = (11, 0, 1)

Find a sequence:

- Choose 1 (A[1]=1, B[1]=11) → A: 1, B: 11 → not equal
- Choose 2 (A[2]=10, B[2]=0) → A: 10, B: 0 → not equal
- Try sequence 1, 2:
  A: 1 + 10 = 110
  B: 11 + 0 = 110 → Match found!

Thus, the sequence [1, 2] satisfies PCP.

Q18. Define formally type 0, type 1, type 2 and type 3 grammar. Show the corresponding automata for each class. (4 Marks)

Type 0 Grammar (Unrestricted Grammar):

1. **Definition:** Type 0 grammars have production rules of the form $\alpha \rightarrow \beta$, where α and β are any strings of terminals and non-terminals, with $\alpha \neq \varepsilon$. There are no restrictions on the form of productions.

2. **Important Info:** These grammars are the most general and can describe all languages that a Turing machine can recognize, i.e., recursively enumerable languages.

3. **Corresponding Automata:** Languages generated by Type 0 grammars are recognized by **Turing Machines (TM)**.

Type 1 Grammar (Context-Sensitive Grammar):

1. **Definition:** Type 1 grammars have production rules of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$, where A is a non-terminal, α and β are strings of terminals and non-terminals, and γ is a non-empty string ($|\gamma| \geq 1$).

2. **Important Info:** These grammars are **length-increasing**, meaning that the length of the output string is at least the same or greater than the input string. They generate **context-sensitive languages**.

3. **Corresponding Automata:** Recognized by **Linear Bounded Automata (LBA)**, a restricted form of Turing machines with bounded tape size.

Type 2 Grammar (Context-Free Grammar):

1. **Definition:** Type 2 grammars have production rules where a single non-terminal leads to a string of terminals and/or non-terminals, i.e., $A \rightarrow \gamma$, where A is a non-terminal and $\gamma \in (V \cup T)^*$.

2. **Important Info:** These grammars generate **context-free languages** and are used widely in programming language design and parsers.

3. **Corresponding Automata:** Recognized by **Pushdown Automata (PDA)**, which use a stack for memory.

Type 3 Grammar (Regular Grammar):

1. **Definition:** Type 3 grammars have very restricted production rules, either of the form A → aB or A → a, where 'a' is a terminal and B is a non-terminal.

2. **Important Info:** These grammars generate **regular languages**, which are the simplest among the Chomsky hierarchy and are used in text searching, lexical analysis, etc.

3. **Corresponding Automata:** Recognized by **Finite Automata (FA)** — either **Deterministic Finite Automata (DFA)** or **Non-Deterministic Finite Automata (NFA)**.

Section - D

Q26. Define Turing machine with the various types of Turing machines. Explain each type in brief. (12 Marks)

A **Turing Machine (TM)** is a mathematical model of computation that defines an abstract machine which manipulates symbols on an infinite tape according to a set of rules. It was proposed by **Alan Turing** in 1936 and is capable of simulating any algorithm.
It provides a simple yet powerful model for understanding what can be computed and what cannot.

Formal Definition:

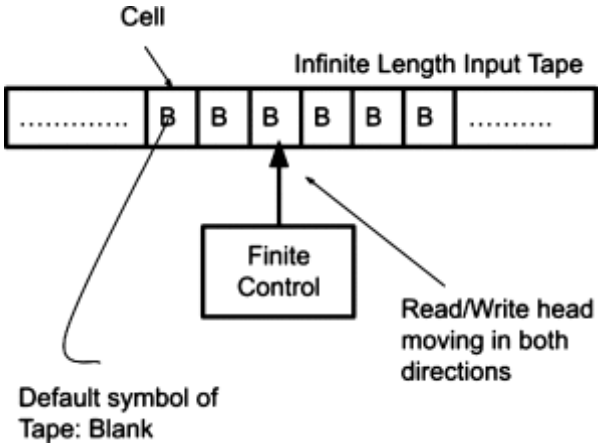
A Turing Machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- **Q** = Finite set of states
- **Σ** = Input alphabet (does not include blank symbol ␣)
- **Γ** = Tape alphabet (finite set of symbols)
- **δ** = Transition function, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $\mathbf{q_0}$ = Start state (initial state)
- **B** = initial tape symbol

- **F** = set of final states

Working Principle:

- The machine reads the tape symbol under the head.
- Based on the symbol and the current state, it writes a new symbol, moves the head **left (L)** or **right (R)**, and transitions to a new state.
- The machine continues until it reaches an accepting or rejecting state.



Types of Turing Machines:

1. Deterministic Turing Machine (DTM):

- In DTM, for each combination of state and tape symbol, the transition function specifies **exactly one action**.
- The behavior of the machine is completely determined at every step.
- **Example:** Standard single-tape Turing machines.

2. Non-Deterministic Turing Machine (NDTM):

- In NDTM, for a given state and tape symbol, there **can be multiple possible actions**.
- The machine accepts an input if **at least one** sequence of choices leads to an accepting state.
- It is a theoretical model; real machines are deterministic, but NDTMs are useful in complexity theory.
- **Note:** NDTMs are **equivalent** to DTMs in terms of computational power.

3. Multi-Tape Turing Machine:

- These TMs have **more than one tape** and **more than one head**, each head can independently read/write on its tape.
- Speeds up computation as multiple operations can be performed simultaneously.
- Despite being faster, they are equivalent to single-tape TMs (can simulate each other).

4. Multi-Track Turing Machine:

- A single tape is divided into **multiple tracks**, and each cell on the tape holds a tuple of symbols (one from each track).
- The head reads/writes to all tracks at once.
- Simplifies programming complex machines but does not increase computational power.

5. Two-Way Infinite Tape Turing Machine:

- The tape is infinite in **both left and right directions**.
- In standard TM, the tape is infinite only to the right.
- Again, computational power remains the same.

6. Turing Machine with Stay Option:

- Apart from moving left (L) and right (R), the machine can also **stay in the same cell (S)** after a transition.
- Useful for specific algorithms but does not change the fundamental capabilities.

7. Offline Turing Machine:

- A multi-tape TM where the **input tape is read-only** and only working tapes are writable.
- Used in analyzing complexity classes.

8. Oracle Turing Machine:

- An abstract machine that has access to an **oracle** — a black box that can instantly solve a particular problem.
- Used in theoretical computer science, especially in **complexity theory**.

Key Points to Remember:

- **All these types** are equivalent in terms of computational power (they recognize the same class of languages: recursively enumerable languages).
- Some types (like multi-tape TMs) provide **efficiency** (faster computation) but not additional power.
- Turing Machines are fundamental in understanding **decidability**, **computability**, and **complexity theory**.